Métodos Numéricos: Cambio de bases

Javier Alexander López Contreras

4 de febrero de 2026

1. Introducción

El cambio de bases de decimal a binario se ha usado a lo largo de la historia para poder comunicarnos con las computadoras. El binario es un sistema numérico que se basa en la combinación de 2 números (0 y 1) y se usa para dar instrucciones o ingresar datos a una computadora. Por otro lado, el lenguaje octal es una agrupación de 8 números (0 al 7) que se utiliza para poder leer las máquinas, agrupando números binarios de 3 en 3, lo que facilita la lectura del codigo binario.

El objetivo de este reporte es explicar los fundamentos matemáticos que rigen el cambio de base.

2. Algoritmo y explicación del método

El método utilizado es el de **Divisiones Sucesivas**. La lógica es idéntica para ambas bases, cambiando el divisor, 2 para binario y 8 para octal (n).

2.1. Metodología

- Se divide el número decimal entre la base n.
- El residuo resultante se convierte en el dígito de la nueva base (se empieza por la derecha).
- El cociente de ulitiza para la siguiente división.
- El proceso termina cuando el cociente sea 0.

EJEMPLO:

A binario: Se divide sucesivamente entre 2. El resultado es una cadena larga de ceros y unos.

A octal: Se diide sucesivamente entre 8. El proceso es más corto y genera dígitos del 0 al 7.

En lugar de dividir uno por uno de manera manual, aplicaremos una sumatoria de la siguiente forma.

$$N_{10} = \sum_{i=0}^{n-1} d_i \cdot B^i \tag{1}$$

N_{10}**:** El número resultante en sistema decimal.

d_i**:** El valor del dígito en la posición i.

B**:** La base de origen (2 para binario, 8 para octal).

i**:** La posición del dígito (comenzando en 0 desde la derecha).

n**:** La cantidad total de dígitos del número original.

3. Análisis del código

El programa en Python para realizar estas operaciones se llama

`Calculadora_conversiones.py` y funciona sin bibliotecas externas, lo que demuestra que la aritmética computacional se basa en procesos lógicos finitos

El código comienza con una función principal que gestiona un bucle infinito (`while True`). Esto garantiza que el programa sea capaz de hacer varias operaciones sin cerrarse. Se incluyó una validación de las opciones para mitigar errores de entrada.

```python
def mostrar_menu():
    while True:
        print("\n" + "=" * 50)
        print("Programa que convierte decimal, binario y octal")
        print("=" * 50)
        print("[1] Decimal -> Binario")
        print("[2] Decimal -> Octal")
        print("[3] Binario -> Decimal")
        print("[4] Binario -> Octal")
        print("[5] Octal   -> Decimal")
        print("[6] Octal   -> Binario")
        print("[7] Salir")

        opcion_base = input("Ingrese una opción (1-7) y luego presione enter: ")
        print("=" * 50)

        if opcion_base == '7':
            print("Saliendo... ¡Hasta luego!")
            break

        if opcion_base not in ['1', '2', '3', '4', '5', '6']:
            print(">>> ERROR: Opción no válida. Intente de nuevo.")
            continue

        num_decimal = 0.0
        if opcion_base in ['3', '5']: base_actual = "Decimal"
        elif opcion_base in ['1', '6']: base_actual = "Binario"
        else: base_actual = "Octal"
```

Figura 1: Interfaz del menú del programa.

El programa convierte primero el valor a una representación interna en `punto flotante`.

- Para binario y octal: Se aplica el Teorema Fundamental de la Numeración mediante un ciclo que recorre la parte entera y la parte decimal multiplicando cada digito por B^i

- Manejo de Fracciones: El código permite la entrada de fracciones, lo cual previene la perdida de información antes de inicial el cálculo.

```python
while True:
k to collapse the range.
        if opcion_base in ['1', '2']:
            entrada = input("Ingrese número decimal: ")
            if "/" in entrada:
                p = entrada.split("/"); num_decimal = int(p[0]) / int(p[1])
            else:
                num_decimal = float(entrada)
            break

        if opcion_base in ['3', '4']:
            entrada = input("Ingrese número binario: ")
            ent_calc = entrada[1:] if entrada.startswith("-") else entrada
            if not all(c in '01.' for c in ent_calc):
                print(">>> Ey, ese no es un número binario válido.")
                continue
            p_e, p_d = (ent_calc.split(".") if "." in ent_calc else (ent_calc, ""))
            val_abs = int(p_e, 2) if p_e else 0
            for i, bit in enumerate(p_d):
                val_abs += int(bit) * (2 ** -(i + 1))
            num_decimal = -val_abs if entrada.startswith("-") else val_abs
            break

        if opcion_base in ['5', '6']:
            entrada = input("Ingrese número octal y luego presione enter: ")
            ent_calc = entrada[1:] if entrada.startswith("-") else entrada
            if not all(c in '01234567.' for c in ent_calc):
                print(">>> Ese no es un número octal válido.")
                continue
            p_e, p_d = (ent_calc.split(".") if "." in ent_calc else (ent_calc, ""))
            val_abs = int(p_e, 8) if p_e else 0
            for i, d in enumerate(p_d):
                val_abs += int(d) * (8 ** -(i + 1))
            num_decimal = -val_abs if entrada.startswith("-") else val_abs
            break
    except ValueError:
        print(">>> Formato incorrecto.")
```

Figura 2: Gestión de datos de entrada.

Una vez que el número está en la variable num_decimal, el programa puede transformarlo a cualquier base solicitada mediante el método de `divisiones sucesivas` para la parte entera y `multiplicaciones sucesivas` para la parte decimal.

Se observa el uso de round(num_decimal, 10). Esto es para limpiar el ruido de la precisión que ocurre cuando un número real no puede ser representado **exactamente** como un número de punto flotante binario.

Para las partes decimales infinitas, se implemento un **límite de 32 iteraciones** (len(res_decimal) < 32), haciendo un error de entruncamiento controlado para evitar bucles infinitos en números no exactos.

```python
while True:
    resultado_final = ""
    if base_actual == "Decimal":
        # Redondeo a 10 decimales para limpiar el ruido de precisión (0.9999999999 -> 1.0)
        # rstrip elimina ceros innecesarios a la derecha
        resultado_final = format(round(num_decimal, 10), '.10f').rstrip('0').rstrip('.')

    elif base_actual == "Binario":
        entero = int(abs(num_decimal)); decimal = abs(num_decimal) - entero
        res_e = bin(entero).replace("0b", "") if entero != 0 else "0"
        res_decimal = ""
        temp_d = decimal
        while temp_d > 0 and len(res_decimal) < 32:
            temp_d *= 2; digito = int(temp_d)
            res_decimal += str(digito); temp_d -= digito
        resultado_final = ("-" if num_decimal < 0 else "") + res_e + ("." + res_decimal if res_decimal else "")

    elif base_actual == "Octal":
        entero = int(abs(num_decimal)); decimal = abs(num_decimal) - entero
        res_e = oct(entero).replace("0o", "") if entero != 0 else "0"
        res_decimal = ""
        temp_d = decimal
        while temp_d > 0 and len(res_decimal) < 32:
            temp_d *= 8; digito = int(temp_d)
            res_decimal += str(digito); temp_d -= digito
        resultado_final = ("-" if num_decimal < 0 else "") + res_e + ("." + res_decimal if res_decimal else "")

    print("-" * 40)
    print(f"El resultado en {base_actual} es: {resultado_final}")
    print("-" * 40)

    resp = input("¿Quieres convertir este mismo número a otra base? (s/n): ").lower()

    if resp == 's':
        opciones = ["Decimal", "Binario", "Octal"]
        opciones.remove(base_actual)
        print("\nSeleccione la nueva base y luego presione enter:")
        print(f"[1] {opciones[0]}")
        print(f"[2] {opciones[1]}")
        nueva_op = input("Opción: ")
        if nueva_op == '1': base_actual = opciones[0]
        elif nueva_op == '2': base_actual = opciones[1]
        else: print(">>> Opción inválida.")
        continue
    break
```

Figura 3: Reconversión de número ingresado.

Para finalizar, el código gestiona la terminación del programa y la persistencia de los datos. Permite al usuario decidir si desea realizar una nueva conversión con el mismo valor aprovechando el almacenamiento de la variable num_decimal o reiniciar el proceso para un nuevo valor.

También asegura que el programa se cierre solo cuando el usuario lo indique mediante la opción de salida.

```python
continuar_programa = True
while True:
    answer = input("\n¿Hacer otra conversión con un número nuevo? (s/n): ").lower()
    if answer == "s":
        break
    elif answer == "n":
        print("Gracias por usar la calculadora :)")
        continuar_programa = False
        break
    else:
        print(">>> Escribe 's' o 'n'.")

if not continuar_programa:
    break
```

Figura 4: Pregunta sobre seguir con el proceso de conversión o salir.

4. Resultados y Análisis

Al convertir el número decimal 0.1 a binario, observamos que el programa genera una cadena finita de dígitos. Matemáticamente, ese valor tiene un desarrollo infinito, pero debido a la aritmética de punto flotante es finita, el software aplica un **error de truncamiento** al detenerse en el bit 32, por lo que no todos los números decimales pueden representarse exactamente en la memoria de la computadora

```
[1] Decimal -> Binario
[2] Decimal -> Octal
[3] Binario -> Decimal
[4] Binario -> Octal
[5] Octal   -> Decimal
[6] Octal   -> Binario
[7] Salir
Ingrese una opción (1-7) y luego presione enter: 1
==============================================
Ingrese número decimal: 0.1
----------------------------------------
El resultado en Binario es: 0.00011001100110011001100110011001
----------------------------------------
```

Figura 5: Error de truncamiento por números decimales.

Se ingresó el valor fraccionario 4/3 para evaluar la precisión del sistema. El programa muestra un resultado de 1.3333333333. Al compararlo con la aritmética real, existe una pérdida de exactitud ya que la computadora solo toma una cantidad limitada de cifras significativas. Esto ocurre por querer representar un número real infinito en un sistema de aritmética computacional finito.

```
Ingrese número decimal: 4/3
----------------------------------------
El resultado en Binario es: 1.01010101010101010101010101010101
----------------------------------------
¿Quieres convertir este mismo número a otra base? (s/n): s

Seleccione la nueva base y luego presione enter:
[1] Decimal
[2] Octal
Opción: 1
----------------------------------------
El resultado en Decimal es: 1.3333333333
----------------------------------------
```

Figura 6: Error de exactitud por cifras significativas.

5. Conclusiones

Mediante el desarrollo de este programa y su experimentación, se ha comprobado de manera práctica que existe una diferencia fundamental entre la aritmética real y la aritmética computacional, debido a que el conjunto de los números de punto flotante es un subconjunto propio y finito de los números reales. Esta naturaleza de la computadora genera una distribución no uniforme de los números en la recta real, concentrándolos cerca del origen y dejando "huecos"que dan pie al error.

Al operar con fracciones como la vista anteriormente, el programa debe aplicar obligatoriamente un error de truncamiento para ajustarse a los bits disponibles, lo que da la perdida de precisión por las cifras significativas.

Aunque las herramientas digitales son indispensables en muchos ámbitos de la vida, su uso requiere una comprensión muy profunda de las fuentes de error para garantizar que los resultados obtenidos sean satisfactorios y confiables dentro de un entorno de cálculo aproximado.

Referencias

[1] Nieves, A. y Domínguez, F. C. *Métodos numéricos aplicados a la ingeniería*. México: Grupo Editorial Patria. (Referencia al material de la Unidad 1.2: Aritmética de Punto Flotante y Errores).

[2] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic (IEEE 754)*. Estándar técnico sobre la representación de números binarios en computadoras.

[3] Python Software Foundation. *The Python Tutorial: Floating Point Arithmetic, Issues and Limitations*. Disponible en: `https://docs.python.org/3/tutorial/floatingpoint.html`

[4] Moler, C. *Numerical Computing with MATLAB*. Society for Industrial and Applied Mathematics (SIAM). (Fuente de consulta para los conceptos de cancelación catastrófica y precisión finita).